BCS703: CNS MODULE 3

Question 1:

Explain the symmetric key distribution using asymmetric encryption.

Answer

Introduction

Asymmetric encryption can be used to **securely distribute a symmetric secret key** between two communicating parties.

Public-key cryptosystems are computationally inefficient for encrypting large amounts of data. Therefore, they are generally used to **encrypt and distribute a symmetric session key**, while the actual communication is carried out using conventional symmetric encryption.

The symmetric key used for a particular communication session is called the **session key Ks**.

Simple Secret Key Distribution

Consider two users:

- **A** — sender
- **B** — receiver

The following procedure is used.

Step 1: A generates a key pair

A generates an asymmetric key pair:

$$\boxed{\{PU_a, PR_a\}}$$

where:

- PU_a = A's public key
- PR_a = A's private key

A sends the public key and its identifier to B:

$$\boxed{PU_a \parallel ID_A}$$

Step 2: B generates a symmetric session key

B generates a secret symmetric key:

$$\boxed{K_s}$$

B encrypts this secret key using A's public key:

$$\boxed{E(PU_a, K_s)}$$

B sends the encrypted session key to A.

Step 3: A recovers the session key

A receives:

$$E(PU_a, K_s)$$

A uses its private key PR_a to decrypt it:

$$\boxed{D(PR_a, E(PU_a, K_s)) = K_s}$$

Only A possesses PR_a, so A can recover the session key.

Step 4: Symmetric communication

Now both A and B possess the same secret key:

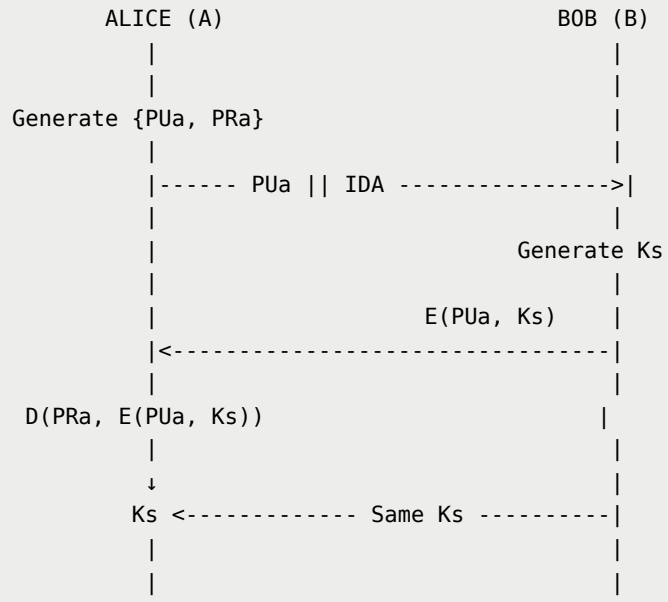
$$\boxed{K_s}$$

They can use K_s with a conventional symmetric encryption algorithm to securely communicate.

After the communication is completed, both parties discard the session key.

Diagram

Draw this diagram in the examination:



```
                    |<==== Symmetric Encryption ======>|
                    |          using Ks                |
                    |                                  |
                    ↓                                  ↓
               Secure Communication
```

Mathematical Representation

The complete process can be represented as:

Public-key phase

$$K_s \xrightarrow{PU_a} E(PU_a, K_s)$$

B sends the encrypted session key to A.

Recovery phase

$$E(PU_a, K_s) \xrightarrow{PR_a} K_s$$

Therefore:

$$D(PR_a, E(PU_a, K_s)) = K_s$$

Communication phase

After establishing K_s:

$$M \xrightarrow{K_s} C$$

and the receiver decrypts:

$$C \xrightarrow{K_s} M$$

Thus, **asymmetric encryption is used only for secure key distribution**, while symmetric encryption is used for the actual data communication. BCS703-module-3-textbook

Advantages

1. **Secure key distribution:** The symmetric key is transmitted in encrypted form.
2. **Efficient data encryption:** Symmetric encryption is much more suitable for large amounts of data.

3. **Temporary session key:** The session key exists only for the communication session.
4. **Reduced risk of key compromise:** The session key is discarded after communication.
5. **Combines advantages of both cryptosystems:** Asymmetric encryption solves the key-distribution problem, while symmetric encryption provides efficient data encryption.

Security Consideration

The simple protocol described above is secure against **eavesdropping**, because the session key Ks is transmitted in encrypted form.

However, it is vulnerable to a **man-in-the-middle attack** if an attacker can intercept messages and substitute or relay them. Therefore, authentication of public keys is required in a practical system.

Question 2:

Explain the role of cryptographic hash functions in message authentication with a neat diagram.

Answer

1. Introduction

Message authentication is a mechanism or service used to verify the **integrity of a message**. It assures that the data received is exactly the data that was sent, i.e., there has been no **modification, insertion, deletion, or replay** of the message.

When a hash function is used for message authentication, the hash value is commonly called a **message digest**.

A cryptographic hash function takes a message of variable length and produces a **fixed-length hash value**:

$$\boxed{h = H(M)}$$

where:

- M = message
- H = cryptographic hash function
- h = hash value/message digest

---

2. Role of Hash Function in Message Authentication

The basic idea is that the sender calculates a hash value from the message and sends the message along with the hash value.

At the Sender

The sender computes:

$$\boxed{h = H(M)}$$

The sender then transmits:

$$\boxed{M \parallel h}$$

where $\parallel$ represents concatenation.

At the Receiver

The receiver receives the message and hash value.

The receiver independently calculates:

$$\boxed{h' = H(M)}$$

The receiver then compares the calculated hash h' with the received hash h.

- If

$$h' = h$$

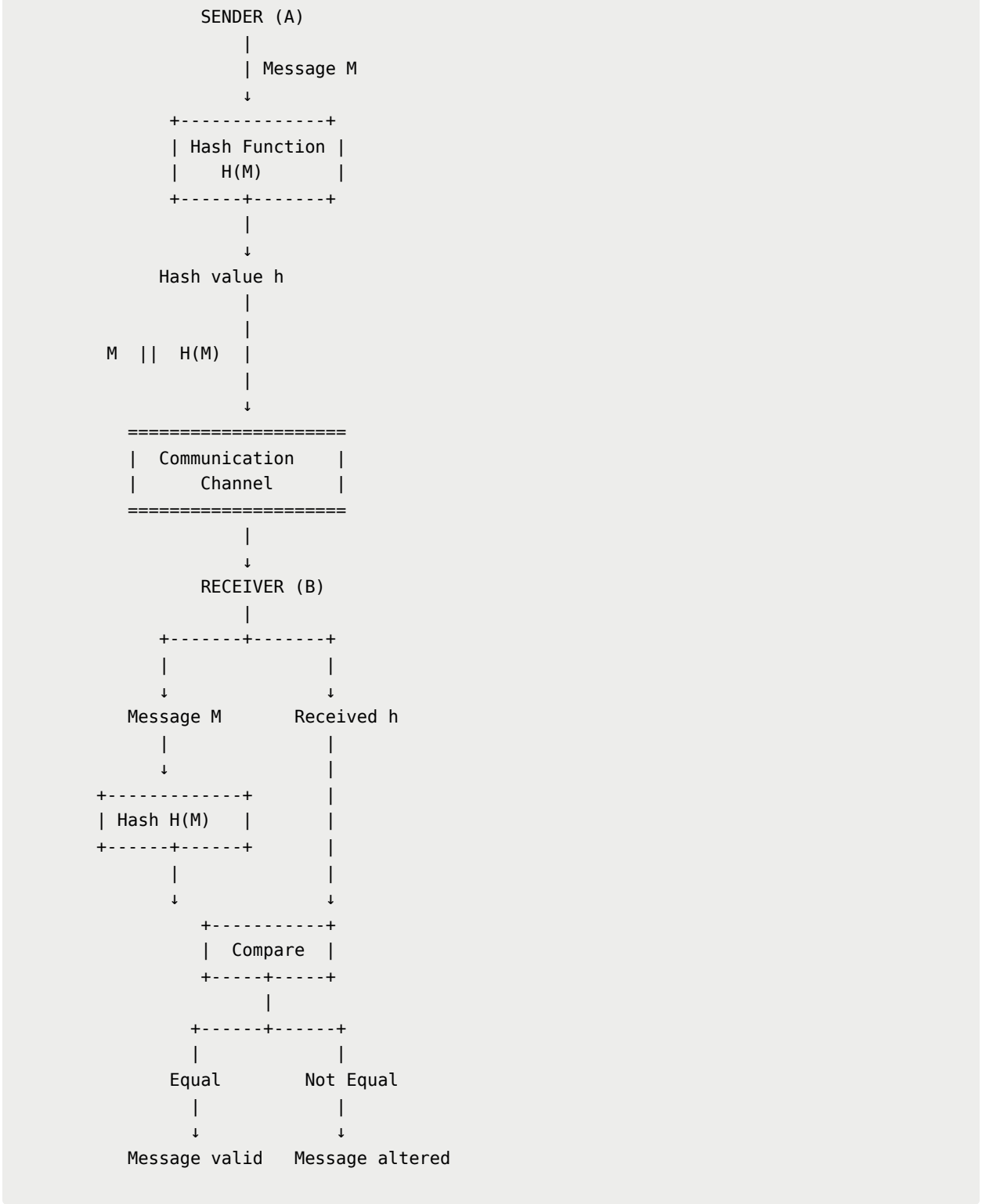
the message is accepted as having the expected integrity.

- If

$$h' \neq h$$

the receiver knows that the message or associated hash value has been altered. BCS703-module-3-textbook

3. Basic Diagram



4. Need to Protect the Hash Value

Simply attaching an ordinary hash value to the message is **not sufficient for message authentication**.

Suppose the sender sends:

$$M \parallel H(M)$$

An attacker can intercept the message, modify M to M', calculate:

$$H(M')$$

and send:

$$M' \parallel H(M')$$

The receiver calculates $H(M')$, obtains the same value, and may fail to detect the modification.

Therefore, the hash value must itself be **protected** so that an attacker cannot replace both the message and its corresponding hash value. BCS703-module-3-textbook

---

5. Hash Function with Secret Value

One method of using a hash function for message authentication is to use a **shared secret value** S.

Both communicating parties know S, but the secret value is not transmitted.

The sender calculates:

$$\boxed{h = H(M \parallel S)}$$

and sends:

$$\boxed{M \parallel H(M \parallel S)}$$

The receiver, who also knows S, calculates:

$$\boxed{h' = H(M \parallel S)}$$

and compares it with the received hash value.

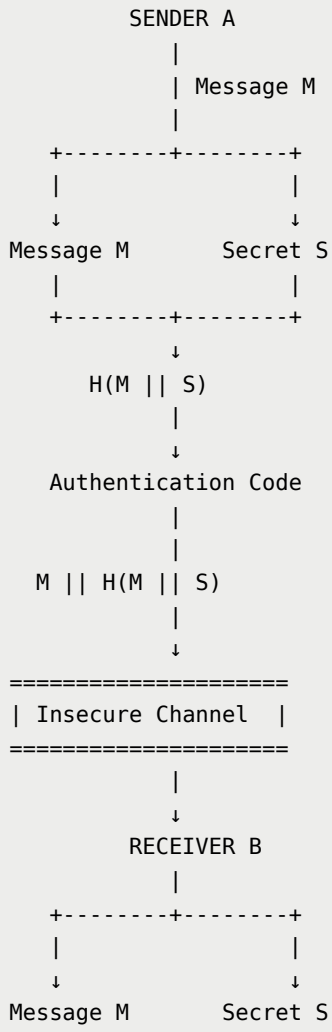
If:

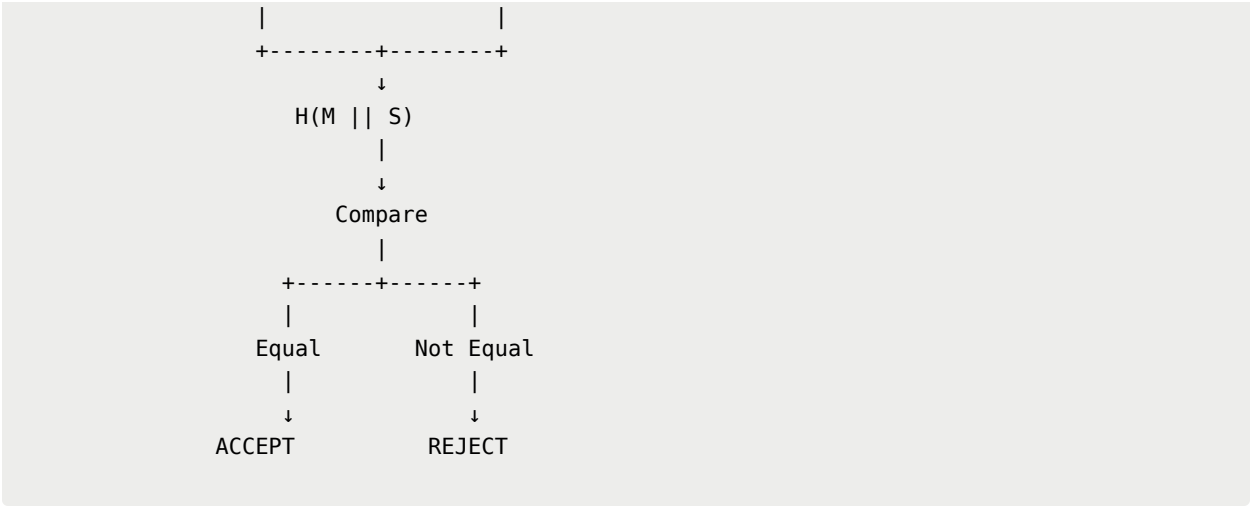
$$\boxed{h = h'}$$

the message passes the authentication check.

Because the secret value S is not sent, an attacker who does not know S cannot generate a valid hash for a modified message. BCS703-module-3-textbook

6. Diagram Using Secret Value





7. Message Authentication Code (MAC)

More commonly, message authentication is achieved using a **Message Authentication Code (MAC)**, also called a **keyed hash function**.

A MAC takes:

- a secret key K
- a data block M

and produces a fixed-size authentication value:

$$\boxed{MAC = C_K(M)}$$

or conceptually:

$$\boxed{MAC = H(K, M)}$$

The sender sends the message together with its MAC.

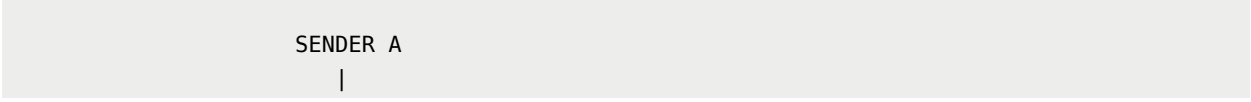
The receiver applies the MAC function using the same secret key and compares the result with the received MAC.

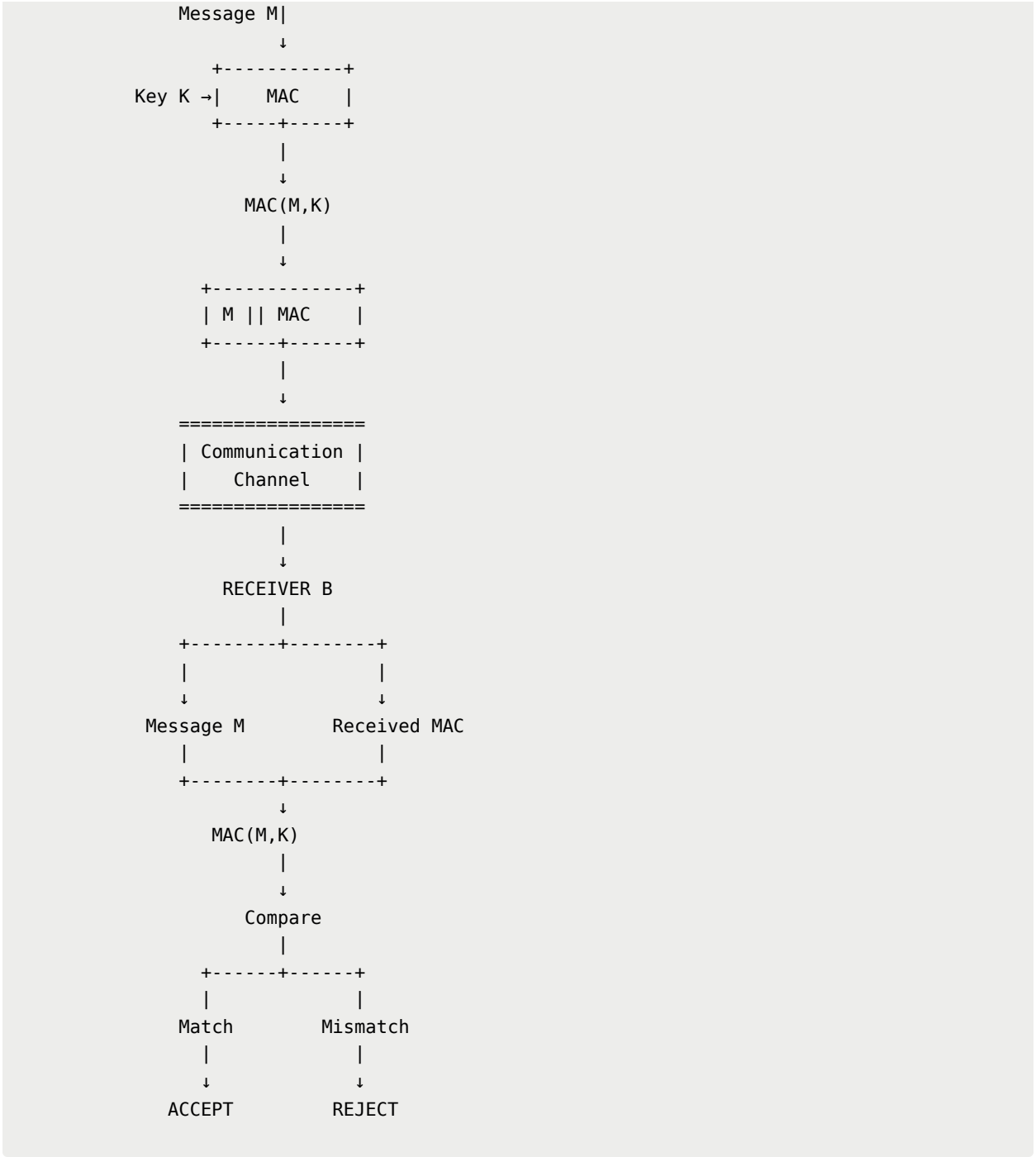
If the values match, the message has passed the authentication check.

An attacker who modifies the message cannot generate the correct MAC without knowing the secret key.

BCS703-module-3-textbook

8. MAC-Based Message Authentication Diagram




Message M
Key K
MAC
MAC(M,K)
M || MAC
Communication Channel
RECEIVER B
Message M
Received MAC
MAC(M,K)
Compare
Match
Mismatch
ACCEPT
REJECT

9. Different Ways of Using Hash Functions for Message Authentication

A hash function can be used for message authentication in several ways.

Method 1: Encrypt Message + Hash

The message and its hash code are encrypted using symmetric encryption:

$$\boxed{E_K[M \parallel H(M)]}$$

This provides **message authentication and confidentiality**. BCS703-module-3-textbook

Method 2: Encrypt Only Hash Code

Only the hash value is encrypted:

$$\boxed{M \parallel E_K[H(M)]}$$

This reduces the processing requirement when confidentiality of the message is not required.
BCS703-module-3-textbook

Method 3: Hash with Secret Value

The sender calculates:

$$\boxed{H(M \parallel S)}$$

and sends the message along with the resulting hash.

The receiver uses the same secret value S to recompute the hash and compare the results.
BCS703-module-3-textbook

10. Security Provided by Hash-Based Authentication

Hash functions contribute to message authentication mainly by providing **message integrity**.

If the message changes from M to M', the resulting hash is expected to change:

$$H(M) \neq H(M')$$

Therefore, modification of the message can be detected when the authentication value is properly protected.

For secure cryptographic hashing, collisions should be computationally difficult to find. A **collision** occurs when:

$$M_1 \neq M_2$$

but:

$$\boxed{H(M_1) = H(M_2)}$$

Collisions are undesirable when hash functions are being used for data integrity. BCS703-module-3-textbook

---

11. Advantages

1. **Message integrity:** Detects unauthorized modification of data.
2. **Efficient processing:** Only a fixed-size hash/MAC needs to be processed for authentication.
3. **Suitable for variable-length messages:** A message of variable length can produce a fixed-length hash value.
4. **Authentication with secret key:** A MAC prevents an attacker without the secret key from generating a valid authentication value.
5. **Can be combined with encryption:** Hash-based authentication can also be combined with encryption when confidentiality is required. BCS703-module-3-textbook